

IF25-22017

PENGEMBANGAN APLIKASI MOBILE

PERTEMUAN 9

Integrasi AI API

OpenAI, Gemini, Claude dan Prompt Engineering

Program Studi Teknik Informatika
Institut Teknologi Sumatera
Tahun Akademik Genap 2025/2026



LEARNING OBJECTIVES

- 1 Memahami cara kerja Large Language Models (LLMs)
- 2 Mengintegrasikan OpenAI API ke aplikasi KMP
- 3 Mengintegrasikan Google Gemini API ke aplikasi KMP
- 4 Menerapkan teknik Prompt Engineering yang efektif
- 5 Membangun fitur AI-powered dalam aplikasi mobile

01

PENGENALAN AI DAN LLMs

Memahami Large Language Models

Apa itu Large Language Model (LLM)?

LLM adalah model AI yang dilatih pada miliaran teks untuk memahami dan menghasilkan bahasa manusia. Model ini dapat:

- Menjawab pertanyaan
- Menerjemahkan bahasa
- Merangkum dokumen
- Menulis kode
- Menganalisis sentimen
- Membuat konten kreatif

GPT-4o

OpenAI

Multimodal, reasoning kuat

Gemini 2.0

Google

Multimodal, gratis tier

Claude 3.5

Anthropic

Coding dan analisis

Llama 3

Meta

Open source, self-host

Arsitektur AI API dalam Aplikasi Mobile



1

User Input

User mengetik pertanyaan atau perintah di aplikasi

2

Prepare Request

Aplikasi membuat prompt dan request ke API

3

HTTP Request

Ktor Client mengirim POST request dengan API key

4

AI Processing

LLM memproses prompt dan generate response

5

Display Result

Response ditampilkan ke user dalam UI

02

OPENAI API

GPT-4o, GPT-4o-mini, dan Chat Completions

Setup OpenAI API

1

Buat Akun OpenAI

Daftar di platform.openai.com

2

Generate API Key

Settings → API Keys → Create new secret key

3

Add Credits

Billing → Add payment method → Add credits (\$5-10)

4

Simpan API Key

Simpan di `local.properties` (JANGAN commit ke Git!)

SECURITY WARNING

JANGAN pernah commit API key ke repository!
Gunakan `local.properties` atau `environment variables`

API Key Management

Menyimpan API key dengan aman

```
// local.properties (JANGAN commit file ini!)
OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxxx

// build.gradle.kts (compose App)
android {
    defaultConfig {
        val localProps = rootProject.file("local.properties")
        val properties = java.util.Properties()
        if (localProps.exists()) {
            properties.load(localProps.inputStream())
        }
        buildConfigField("String", "OPENAI_API_KEY",
            "\"${properties.getProperty("OPENAI_API_KEY", "")}\"")
    }
    buildFeatures { buildConfig = true }
}

// Config.kt (commonMain) - expect/actual pattern
expect object ApiConfig {
    val openAiApiKey: String
}

// Config.android.kt (androidMain)
actual object ApiConfig {
    actual val openAiApiKey: String = BuildConfig.OPENAI_API_KEY
}
```

OpenAI Data Models

Request dan Response DTOs

```
// OpenAI Request/Response Models
@Serializable
data class ChatCompletionRequest(
    val model: String = "gpt-4o-mini",
    val messages: List<ChatMessage>,
    @SerializedName("max_tokens") val maxTokens: Int = 1000,
    val temperature: Double = 0.7
)

@Serializable
data class ChatMessage(
    val role: String, // "system", "user", "assistant"
    val content: String
)

@Serializable
data class ChatCompletionResponse(
    val id: String,
    val choices: List<Choice>,
    val usage: Usage
)

@Serializable
data class Choice(
    val index: Int,
    val message: ChatMessage,
    @SerializedName("finish_reason") val finishReason: String
)

@Serializable
data class Usage(
    @SerializedName("prompt_tokens") val promptTokens: Int,
    @SerializedName("completion_tokens") val completionTokens: Int,
    @SerializedName("total_tokens") val totalTokens: Int
)
```

OpenAI Service

Implementasi service untuk chat completions

```
class OpenAIService(private val client: HttpClient) {

    private val baseUrl = "https://api.openai.com/v1"

    suspend fun chat(
        messages: List<ChatMessage>,
        model: String = "gpt-4o-mini"
    ): Result<String> = runCatching {

        val request = ChatCompletionRequest(
            model = model,
            messages = messages,
            maxTokens = 1000,
            temperature = 0.7
        )

        val response: ChatCompletionResponse = client.post("$baseUrl/chat/completions") {
            contentType(ContentType.Application.Json)
            header("Authorization", "Bearer ${ApiConfig.openAiApiKey}")
            setBody(request)
        }.body()

        response.choices.first().message.content
    }

    suspend fun askQuestion(question: String): Result<String> {
        val messages = listOf(
            ChatMessage(role = "system", content = "You are a helpful assistant."),
            ChatMessage(role = "user", content = question)
        )
        return chat(messages)
    }
}
```

03

GOOGLE GEMINI API

Gemini 2.0 Flash dan Multimodal AI

Gemini vs OpenAI Comparison

Aspect	OpenAI (GPT)	Google (Gemini)
Free Tier	 No (pay-as-you-go)	 Yes (15 RPM)
Best Model	GPT-4o (\$5/1M tokens)	Gemini 2.0 Flash (free)
Multimodal	 Image, audio, video	 Image, audio, video
Speed	Medium	Fast (Flash)
Context Window	128K tokens	1M tokens
Best For	Complex reasoning	Fast responses, free tier
 Rekomendasi: Gunakan Gemini untuk development (gratis), OpenAI untuk production		

Setup Google Gemini API

1

Buka Google AI Studio

Kunjungi aistudio.google.com

2

Sign in dengan Google

Gunakan akun Google Anda

3

Get API Key

Klik 'Get API key' → Create API key

4

Copy dan Simpan

Simpan API key di `local.properties`

✓ FREE TIER

15 requests/minute, 1500 requests/day
Cukup untuk development dan small apps!

Gemini Data Models

Request dan Response DTOs

```
// Gemini Request/Response Models
@Serializable
data class GeminiRequest(
    val contents: List<Content>,
    val generationConfig: GenerationConfig? = null
)

@Serializable
data class Content(
    val parts: List<Part>,
    val role: String = "user"
)

@Serializable
data class Part(
    val text: String
)

@Serializable
data class GenerationConfig(
    val temperature: Double = 0.7,
    val maxOutputTokens: Int = 1000,
    val topP: Double = 0.95
)

@Serializable
data class GeminiResponse(
    val candidates: List<Candidate>
)

@Serializable
data class Candidate(
    val content: Content,
    val finishReason: String? = null
)
```

Gemini Service

Implementasi service untuk content generation

```
class GeminiService(private val client: HttpClient) {

    private val baseUrl = "https://generativelanguage.googleapis.com/v1beta"
    private val model = "gemini-2.0-flash"

    suspend fun generateContent(prompt: String): Result<String> = runCatching {

        val request = GeminiRequest(
            contents = listOf(
                Content(parts = listOf(Part(text = prompt)))
            ),
            generationConfig = GenerationConfig(
                temperature = 0.7,
                maxOutputTokens = 1000
            )
        )

        val response: GeminiResponse = client.post(
            "$baseUrl/models/$model:generateContent"
        ) {
            contentType(ContentType.Application.Json)
            parameter("key", ApiConfig.geminiApiKey)
            setBody(request)
        }.body()

        response.candidates.first().content.parts.first().text
    }
}
```

Multi-turn Conversation

Menyimpan context percakapan

```
class GeminiChatService(private val client: HttpClient) {

    private val conversationHistory = mutableListOf<Content>()

    suspend fun chat(userMessage: String): Result<String> = runCatching {
        // Add user message to history
        conversationHistory.add(
            Content(parts = listOf(Part(text = userMessage)), role = "user")
        )

        val request = GeminiRequest(contents = conversationHistory.toList())

        val response: GeminiResponse = client.post("...") {
            // ... same as before
        }.body()

        val assistantMessage = response.candidates.first().content

        // Add assistant response to history
        conversationHistory.add(assistantMessage)

        assistantMessage.parts.first().text
    }

    fun clearHistory() {
        conversationHistory.clear()
    }
}
```

04

PROMPT ENGINEERING

Teknik Membuat Prompt yang Efektif

Apa itu Prompt Engineering?

Prompt Engineering adalah seni dan ilmu merancang instruksi yang efektif untuk mendapatkan output yang diinginkan dari AI.

Prompt yang baik = Output yang baik

Prompt yang buruk = Hasil tidak konsisten

Be Specific

Jelaskan dengan detail apa yang diinginkan

Provide Context

Berikan informasi latar belakang yang relevan

Set Format

Tentukan format output (JSON, list, paragraph)

Use Examples

Berikan contoh output yang diharapkan

Good vs Bad Prompts

✗ Bad Prompt

"Buatkan saya aplikasi"

- Terlalu umum
- Tidak ada konteks
- Tidak ada spesifikasi
- Hasil tidak bisa diprediksi

✓ Good Prompt

"Buat fungsi Kotlin untuk validasi email. Return true jika valid, false jika tidak. Gunakan regex."

- Spesifik
- Ada konteks (Kotlin)
- Format output jelas
- Kriteria sukses jelas

Contoh Prompt Patterns:

- Role: "Kamu adalah nutritionist profesional..."
- Task: "Analisis kandungan gizi dari [input]..."
- Format: "Berikan output dalam format JSON dengan field: kalori, protein, lemak"
- Constraint: "Jawab dalam Bahasa Indonesia, maksimal 200 kata"

System Prompts

Mendefinisikan perilaku AI

```
// System Prompt - Menentukan perilaku AI
object SystemPrompts {

    val NUTRITIONIST = """
        Kamu adalah nutritionist profesional dengan pengalaman 10 tahun.

        Tugas: Menganalisis makanan dan memberikan informasi gizi.

        Rules:
        - Selalu berikan informasi dalam Bahasa Indonesia
        - Sertakan estimasi kalori, protein, karbohidrat, lemak
        - Berikan saran kesehatan yang relevan
        - Jika tidak yakin, katakan "estimasi"
        - Format output sebagai JSON

        Contoh output:
        {
            "makanan": "Nasi Goreng",
            "kalori": 300,
            "protein": 8,
            "karbohidrat": 45,
            "lemak": 10,
            "saran": "Tambahkan sayuran untuk serat"
        }
        """.trimIndent()

    val CODE_ASSISTANT = """
        You are a Kotlin/KMP expert.
        Always provide code that is:
        - Idiomatic Kotlin
        - Compatible with KMP (commonMain)
        - Well-documented with KDoc
        """.trimIndent()
}
```

Structured Output (JSON)

Parsing response AI menjadi data class

```
// Meminta AI menghasilkan JSON yang bisa di-parse
suspend fun analyzeFood(foodName: String): Result<NutritionInfo> = runCatching {

    val prompt = """
        Analisis informasi gizi untuk: $foodName

        PENTING: Respond HANYA dengan JSON, tanpa text lain.

        Format:
        {
            "name": "nama makanan",
            "calories": number,
            "protein": number,
            "carbs": number,
            "fat": number,
            "healthTips": ["tip1", "tip2"]
        }
    """.trimIndent()

    val response = geminiService.generateContent(prompt).getOrNull()

    // Parse JSON response
    val cleanJson = response
        .removePrefix("```json")
        .removeSuffix("```")
        .trim()

    Json.decodeFromString<NutritionInfo>(cleanJson)
}

@Serializable
data class NutritionInfo(
    val name: String,
    val calories: Int,
    val protein: Int,
    val carbs: Int,
    val fat: Int,
    val healthTips: List<String>
)
```

05

AI-POWERED FEATURES

Fitur Cerdas untuk Aplikasi Mobile

Ide Fitur AI untuk Aplikasi

Smart Chatbot

Asisten yang menjawab pertanyaan tentang app

Auto-Summary

Merangkum artikel, notes, atau dokumen panjang

Translation

Terjemahan real-time antar bahasa

Data Insights

Analisis dan insight dari data pengguna

Content Generator

Generate caption, email, atau konten kreatif

Smart Search

Pencarian dengan natural language

Recommendations

Rekomendasi personal berdasarkan preferensi

AI Repository

Abstraksi untuk AI features

```
// Repository pattern untuk AI features
interface AIRepository {
    suspend fun chat(message: String): Result<String>
    suspend fun analyzeFood(foodName: String): Result<NutritionInfo>
    suspend fun summarize(text: String): Result<String>
    suspend fun translate(text: String, targetLang: String): Result<String>
}

class AIRepositoryImpl(
    private val geminiService: GeminiService
) : AIRepository {

    override suspend fun chat(message: String): Result<String> {
        return geminiService.generateContent(message)
    }

    override suspend fun summarize(text: String): Result<String> {
        val prompt = """
            Rangkum teks berikut dalam 2-3 kalimat.
            Fokus pada poin-poin utama.

            Teks:
            $text
            """.trimIndent()

        return geminiService.generateContent(prompt)
    }

    // ... other implementations
}
```

AI ViewModel

State management untuk chat

```
class ChatViewModel(
    private val aiRepository: AIRepository
) : ViewModel() {

    private val _uiState = MutableStateFlow(ChatUiState())
    val uiState: StateFlow<ChatUiState> = _uiState.asStateFlow()

    fun sendMessage(message: String) {
        if (message.isBlank()) return

        // Add user message to UI
        _uiState.update { it.copy(
            messages = it.messages + ChatMessage(message, isUser = true),
            isLoading = true
        )}

        viewModelScope.launch {
            aiRepository.chat(message)
                .onSuccess { response ->
                    _uiState.update { it.copy(
                        messages = it.messages + ChatMessage(response, isUser = false),
                        isLoading = false
                    )}
                }
                .onFailure { error ->
                    _uiState.update { it.copy(
                        error = error.message,
                        isLoading = false
                    )}
                }
        }
    }
}

data class ChatUiState(
    val messages: List<ChatMessage> = emptyList(),
    val isLoading: Boolean = false,
    val error: String? = null
)
```

Chat UI

Compose UI untuk chat interface

```
@Composable
fun ChatScreen(viewModel: ChatViewModel) {
    val uiState by viewModel.uiState.collectAsState()
    var inputText by remember { mutableStateOf("") }

    Column(modifier = Modifier.fillMaxSize()) {
        // Messages list
        LazyColumn(
            modifier = Modifier.weight(1f),
            reverseLayout = true
        ) {
            items(uiState.messages.reversed()) { message ->
                ChatBubble(
                    message = message.text,
                    isUser = message.isUser
                )
            }

            if (uiState.isLoading) {
                item { TypingIndicator() }
            }
        }

        // Input field
        Row(modifier = Modifier.padding(8.dp)) {
            OutlinedTextField(
                value = inputText,
                onValueChange = { inputText = it },
                modifier = Modifier.weight(1f),
                placeholder = { Text("Ketik pesan...") }
            )
            IconButton(
                onClick = {
                    viewModel.sendMessage(inputText)
                    inputText = ""
                },
                enabled = inputText.isNotBlank() dan !uiState.isLoading
            ) {
                Icon(Icons.AutoMirrored.Default.Send, "Kirim")
            }
        }
    }
}
```

Chat Bubble Component

Reusable chat bubble composable

```
@Composable
fun ChatBubble(
    message: String,
    isUser: Boolean,
    modifier: Modifier = Modifier
) {
    val bubbleColor = if (isUser) {
        MaterialTheme.colorScheme.primary
    } else {
        MaterialTheme.colorScheme.surfaceVariant
    }

    val textColor = if (isUser) {
        MaterialTheme.colorScheme.onPrimary
    } else {
        MaterialTheme.colorScheme.onSurfaceVariant
    }

    Row(
        modifier = modifier.fillMaxWidth().padding(8.dp),
        horizontalArrangement = if (isUser) Arrangement.End else Arrangement.Start
    ) {
        Surface(
            shape = RoundedCornerShape(
                topStart = 16.dp, topEnd = 16.dp,
                bottomStart = if (isUser) 16.dp else 4.dp,
                bottomEnd = if (isUser) 4.dp else 16.dp
            ),
            color = bubbleColor,
            modifier = Modifier.widthIn(max = 280.dp)
        ) {
            Text(
                text = message,
                color = textColor,
                modifier = Modifier.padding(12.dp),
                style = MaterialTheme.typography.bodyMedium
            )
        }
    }
}
```

Typing Indicator

Animasi loading saat AI memproses

```
@Composable
fun TypingIndicator() {
    Row(
        modifier = Modifier.padding(16.dp),
        horizontalArrangement = Arrangement.spacedBy(4.dp)
    ) {
        repeat(3) { index ->
            val infiniteTransition = rememberInfiniteTransition()
            val alpha by infiniteTransition.animateFloat(
                initialValue = 0.3f,
                targetValue = 1f,
                animationSpec = infiniteRepeatable(
                    animation = tween(600),
                    repeatMode = RepeatMode.Reverse,
                    initialStartOffset = StartOffset(index * 200)
                )
            )

            Box(
                modifier = Modifier
                    .size(8.dp)
                    .alpha(alpha)
                    .background(
                        MaterialTheme.colorScheme.primary,
                        CircleShape
                    )
            )
        }
    }
}

// Usage: Show when AI is "thinking"
if (uiState.isLoading) {
    TypingIndicator()
}
```

06

ERROR HANDLING

Menangani Error dan Edge Cases

Common AI API Errors

401	Unauthorized API key invalid/expired	Check API key configuration
429	Rate Limited Too many requests	Implement retry with backoff
500	Server Error AI service down	Show fallback, retry later
400	Bad Request Invalid prompt/params	Validate input before send
Timeout	Request Timeout Slow response	Increase timeout, show loading

Error Handling

Sealed class untuk AI errors

```
sealed class AIError : Exception() {
    data class RateLimited(val retryAfter: Int) : AIError()
    data class Unauthorized(override val message: String) : AIError()
    data class ServerError(override val message: String) : AIError()
    data class NetworkError(override val message: String) : AIError()
    data class ParseError(override val message: String) : AIError()
}

suspend fun <T> safeAICall(block: suspend () -> T): Result<T> {
    return try {
        Result.success(block())
    } catch (e: ClientRequestException) {
        when (e.response.status.value) {
            401 -> Result.failure(AIError.Unauthorized("Invalid API key"))
            429 -> {
                val retryAfter = e.response.headers["Retry-After"]?.toIntOrNull() ?: 60
                Result.failure(AIError.RateLimited(retryAfter))
            }
            in 500..599 -> Result.failure(AIError.ServerError("Server error"))
            else -> Result.failure(e)
        }
    } catch (e: IOException) {
        Result.failure(AIError.NetworkError("No internet connection"))
    } catch (e: SerializationException) {
        Result.failure(AIError.ParseError("Failed to parse response"))
    }
}
```

Retry with Exponential Backoff

Auto-retry untuk transient errors

```
suspend fun <T> retryWithBackoff(
    times: Int = 3,
    initialDelay: Long = 1000,
    maxDelay: Long = 10000,
    factor: Double = 2.0,
    block: suspend () -> T
): T {
    var currentDelay = initialDelay
    repeat(times - 1) { attempt ->
        try {
            return block()
        } catch (e: Exception) {
            when {
                e is AIError.RateLimited -> {
                    delay(e.retryAfter * 1000L)
                }
                e is AIError.ServerError -> {
                    delay(currentDelay)
                    currentDelay = (currentDelay * factor).toLong().coerceAtMost(maxDelay)
                }
                else -> throw e
            }
        }
    }
    return block() // Last attempt
}

// Usage
val response = retryWithBackoff {
    aiService.generateContent(prompt)
}
```

07

BEST PRACTICES

Tips dan Rekomendasi

Best Practices untuk AI Integration

Security

- Simpan API key di local.properties
- Jangan expose key di client
- Gunakan backend proxy untuk production

UX

- Selalu tampilkan loading state
- Berikan feedback yang jelas
- Handle errors dengan graceful

Performance

- Cache responses jika memungkinkan
- Batasi panjang prompt
- Gunakan streaming untuk long responses

Cost

- Gunakan model yang tepat (mini vs full)
- Monitor usage di dashboard
- Set budget alerts



TUGAS PRAKTIKUM

Tugas: Integrasi AI ke Aplikasi

Integrasikan minimal SATU fitur AI ke aplikasi project Anda:

- Smart chatbot/assistant
- Food/nutrition analysis
- Content summarization
- Translation feature
- Atau fitur AI kreatif lainnya

- ✓ Gunakan Gemini API (gratis) atau OpenAI API
- ✓ Implementasi proper error handling
- ✓ UI yang responsif dengan loading states
- ✓ System prompt yang well-designed
- ✓ Dokumentasi fitur di README



Deadline: Sebelum Pertemuan 10



Submit: Push ke repository GitHub

Rubrik Penilaian Tugas

Kriteria	Bobot	Deskripsi
AI Integration	30%	API calls work, proper service layer
Prompt Engineering	25%	Well-designed system prompt, structured output
Error Handling	20%	Handles errors gracefully, retry logic
UI/UX	15%	Loading states, responsive, good feedback
Code Quality	10%	Clean code, proper architecture

 Bonus Points:

- Multi-turn conversation (+5%)
- Image analysis (+10%)
- Streaming response (+5%)



Resources dan Documentation

OpenAI

- platform.openai.com/docs
- OpenAI Cookbook (GitHub)
- API Reference

Google Gemini

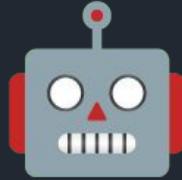
- ai.google.dev/docs
- Google AI Studio
- Gemini API Quickstart

Anthropic Claude

- docs.anthropic.com
- Claude API Reference
- Prompt Library

Learning

- Prompt Engineering Guide
- LangChain Documentation
- AI Integration Patterns



AI is a Tool, Not Magic

*Gunakan AI untuk meningkatkan user experience,
bukan menggantikan good engineering practices.*

Questions? Let's discuss! 